

— STUDI KASUS ARSITEKTUR • PROGRAMMERZAMANNOW

CQRS untuk Product Search

PostgreSQL yang lambat → OpenSearch yang cepat

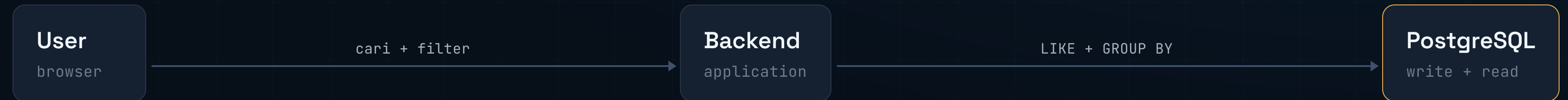
Sebuah studi kasus nyata: pencarian katalog produk e-commerce, dari masalah sederhana sampai solusi CQRS — dan mendemokannya langsung.

● cQRS-demo • keyword + facet @ 1.000.000 produk

Pencarian katalog produk

- Aplikasi punya **katalog produk** — produk, kategori, dan brand.
 - Data awalnya sedikit, jadi **semua terasa cepat**. Masalah baru muncul saat data membesar.
- User mengharapkan pencarian ala toko online
 - **Keyword search** — "sepatu", "wireless mouse"
 - **Filter** — kategori, brand, rentang harga, stok
 - **Facet** — jumlah produk per pilihan: Nike (20), Adidas (30)...
 - **Sorting & pagination**

Satu database saja



- Keyword

```
WHERE LOWER(name) LIKE '%kata%' OR LOWER(description)
LIKE '%kata%'
```

- Facet

Beberapa query `GROUP BY` terpisah — satu per kategori, brand, rentang harga, dan ketersediaan stok.

Lambat di skala besar

WAKTU PROSES QUERY DI BACKEND

JUMLAH PRODUK	KEYWORD SEARCH + FACET
100.000	~0,3 detik
1.000.000	~3,2 detik 🐌

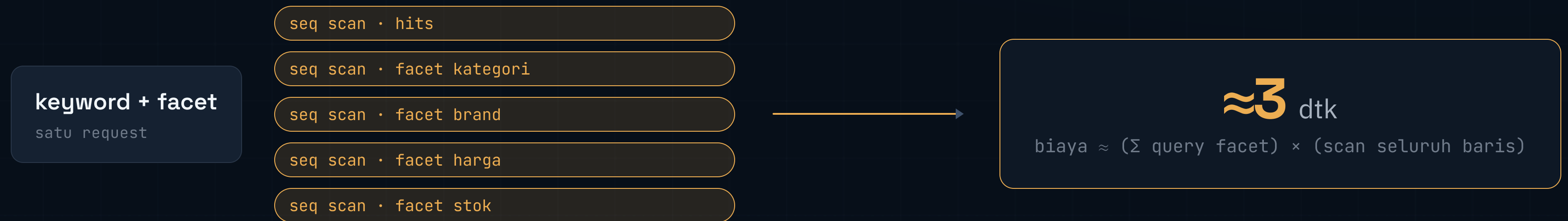
- Makin banyak data → makin lambat, mendekati linear lalu memburuk.

3 detik

Satu kali ketik pencarian di 1 juta data — **fatal untuk UX**. Dan ini baru 1 juta.

Kenapa bisa selambat itu?

- `LIKE '%kata%'` (leading wildcard) → index tidak bisa dipakai → **sequential scan** seluruh tabel.
- **Facet** = beberapa query agregasi `GROUP BY`, masing-masing **full-scan** tersendiri.



Database OLTP dioptimalkan untuk transaksi tulis yang ternormalisasi – **bukan** untuk full-text search + agregasi berat. Kita memakai alat yang salah untuk pekerjaan ini.

Satu model untuk dua kebutuhan

WRITE

Normalisasi, konsistensi, transaksi

OLTP

READ / SEARCH

Full-text, agregasi facet, kecepatan baca

OLAP-ish

Keduanya **tarik-menarik** dalam satu model data yang sama.

Optimasi untuk write $\neq$ optimasi
untuk read.

Command Query Responsibility Segregation

- Pisahkan **model tulis (Command)** dari **model baca (Query)**.
- Tiap sisi dioptimalkan **independen**, pakai teknologi yang paling cocok.



Data ditulis ke satu model, **dibaca dari model lain** yang sudah dioptimalkan untuk baca. CQRS bukan teknologi – tapi pola.

Tidak ada makan siang gratis

+ Manfaat

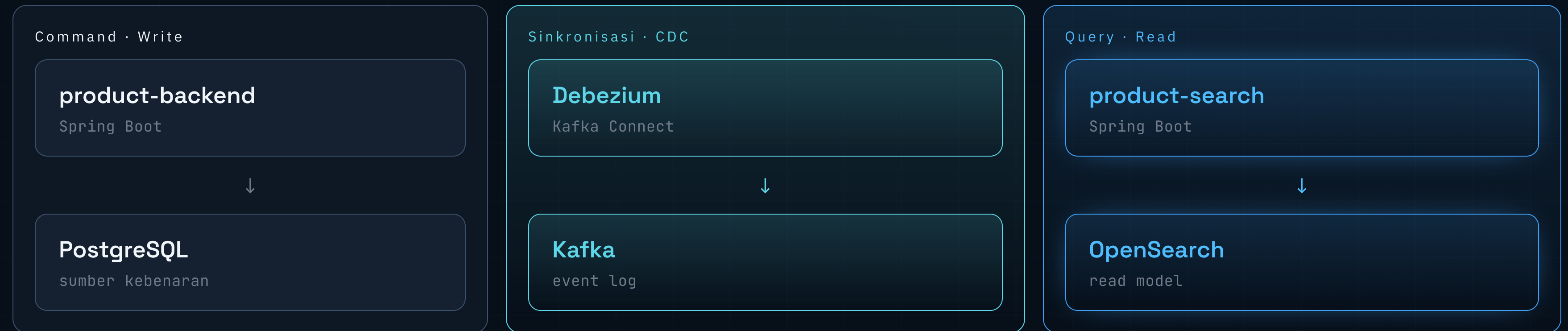
- Read model pakai **engine khusus** (search engine) → baca super cepat.
- Write model tetap **bersih & ternormalisasi**.
- Skala read & write bisa **independen**.

– Konsekuensi

- **Eventual consistency** — read model menyusul write, tidak instan.
- Lebih banyak komponen → **kompleksitas operasional** naik.

Menukar "konsistensi langsung & kesederhanaan" dengan "**kecepatan baca & skalabilitas**". Untuk search, hampir selalu worth it.

Peta besar sistem



PG → WAL logical → Debezium

Kafka → product-search → index OpenSearch

Frontend (nginx + SPA) memanggil `/api/products` ke backend, atau `/api/products/_search` ke product-search – bisa memilih sumber query.

Bagaimana data write sampai ke read model — tanpa backend tahu?

- `product-backend` → PostgreSQL
INSERT / UPDATE produk (commit)
- PostgreSQL → Debezium
perubahan via WAL (logical)
- Debezium → Kafka
publish event (topik produk / brand / kategori)
- Kafka → `product-search`
consume event
- `product-search` → OpenSearch
index / update dokumen → bisa dicari ~1 dtk kemudian

- **Membaca WAL, bukan polling**

Debezium membaca **WAL** PostgreSQL — bukan polling, bukan dual-write.

- **Penulis tak berubah**

`product-backend` tetap menulis ke PostgreSQL seperti biasa.
Write tak peduli ada berapa read model.

CDC adalah "kabel" CQRS — menangkap SEMUA perubahan tanpa mengubah aplikasi penulis.

Kenapa OpenSearch?

ASPEK	POSTGRESQL + LIKE	OPENSEARCH
Keyword substring	seq scan	terindeks
Facet + count	banyak GROUP BY	1 query agregasi
Waktu @ 1 juta	~3 detik	puluhan ms

- **Full-text & substring** cepat karena terindeks — bukan sequential scan.
- **Facet = aggregations native** → hits + semua facet dalam satu query.
- **Relevansi**, typo-tolerance, dan skala horizontal.

n-gram = LIKE tapi cepat

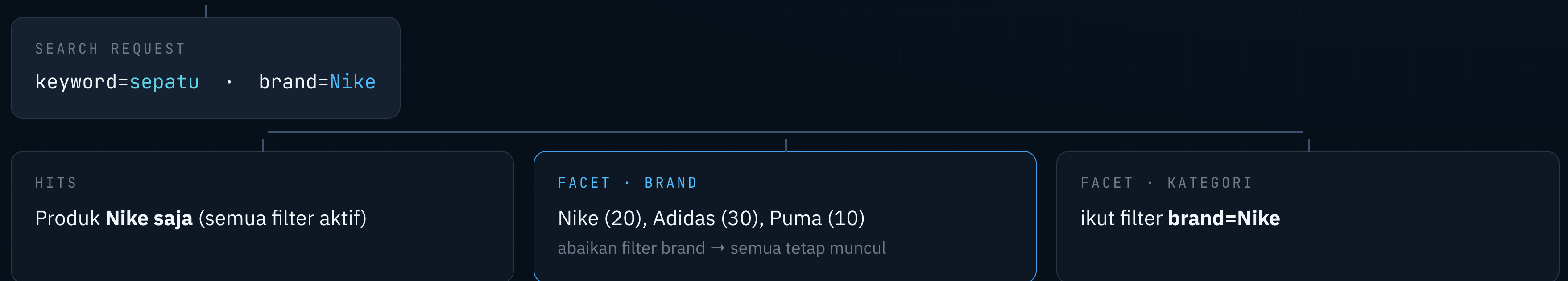
- Field `name` / `description` di-index dengan **analyzer n-gram** (2–3 huruf).
- Query memakai `match_phrase` → n-gram harus **berurutan** → cocok substring panjang berapa pun, tapi **terindeks** → **cepat**.
- Prinsip yang sama dengan `pg_trgm` (trigram) di PostgreSQL.

`"wireless"` → `[wi, wir, ire, rel, ele, les, ess, ...]` `match_phrase`: n-gram harus bersebelahan → substring `"wireless"` ketemu

Hasil demo: total hasil identik dengan **LIKE** PostgreSQL — parity terverifikasi, tanpa false positive — tapi jauh lebih cepat.

Facet drill-down

"Pilih satu brand, brand lain jangan hilang." Satu request → hits + semua facet.



Tiap facet dihitung dengan semua filter aktif **KECUALI filter dimensinya sendiri** — `post_filter` + filter aggregation per dimensi. Di PostgreSQL butuh banyak query terpisah; di OpenSearch jadi satu query elegan.

Eventual consistency



- Cocok untuk search katalog

Lag ~1 detik **sangat dapat diterima**. Tulis di backend → ~1 detik kemudian muncul di `/_search`.

- Butuh konsistensi kuat?

Untuk data seperti **saldo** → tetap baca dari write model. Pilih per use-case.

PostgreSQL vs OpenSearch

FRONTEND

Toggle sumber

format respons identik

GET /api/products

PostgreSQL

~3 dtk @ 1jt ⚠ lambat

GET /api/products/_search

OpenSearch

puluhan ms

Keyword sama, data sama, hasil sama — tapi satu **3 detik**, satu **puluhan milidetik**. Penonton lihat bedanya langsung.

Matches identik, kecepatan jauh berbeda

KEYWORD	MATCHES (PG = 0S)	POSTGRESQL (AVG)	OPENSEARCH (AVG)	SPEED-UP
rubber	79.963	2.930 ms	61 ms	48×
steel	80.065	2.910 ms	44 ms	66×
ergonomic	135.391	2.966 ms	80 ms	37×
wooden	79.641	2.731 ms	39 ms	70×
cotton	78.835	3.020 ms	29 ms	104×
fresh	44.018	2.658 ms	28 ms	95×
handcrafted	38.814	2.842 ms	35 ms	81×

PARITY

Matches **identik** di kedua mesin

RATA-RATA

≈70× lebih cepat

RENTANG

2.658–3.020 ms → 28–80 ms

Setelah warm-up, OpenSearch makin kencang

KEYWORD	MATCHES (PG = OS)	POSTGRESQL (AVG)	OPENSEARCH (AVG)	SPEED-UP
rubber	79.963	2.810 ms	17 ms	165×
steel	80.065	2.913 ms	17 ms	171×
ergonomic	135.391	3.038 ms	23 ms	132×
wooden	79.641	2.680 ms	20 ms	134×
cotton	78.835	2.702 ms	16 ms	169×
fresh	44.018	2.599 ms	22 ms	118×
handcrafted	38.814	2.560 ms	68 ms	38×

PARITY
Matches **tetap identik**

OPENSEARCH • WARM
mayoritas **16–23 ms**

RATA-RATA
≈130× lebih cepat

Query berulang memakai **cache OpenSearch** → turun ke belasan ms; PostgreSQL **LIKE** tetap ~2,6–3,0 dtk karena selalu sequential scan.

Komponen sistem

KOMPONEN	TEKNOLOGI	PERAN
product-backend	Java 25 · Spring Boot 4 · PostgreSQL	Write model + CRUD
Debezium + Kafka	Kafka Connect (CDC) · Kafka (KRaft)	Sinkronisasi write → read
product-search	Spring Boot 4 · Spring Kafka	Projector + Search API
OpenSearch	OpenSearch 3	Read model (search + facet)
product-frontend	SvelteKit · nginx	UI + toggle PG / OS
product-faker	Bun	Generator data demo

► Semua dijalankan via Docker / Podman Compose

Obat untuk masalah spesifik — bukan default

✓ Cocok bila

- Beban **baca jauh lebih berat** / kompleks daripada tulis — search, analitik, dashboard.
- Butuh model baca yang **beda jauh** dari model tulis.
- **Eventual consistency** dapat diterima.

✗ Hindari / tunda bila

- Data kecil & query sederhana — **over-engineering**.
- Butuh **konsistensi kuat** seketika.
- Tim belum siap dengan kompleksitas operasional (Kafka, CDC, dll).

Database yang tepat untuk pekerjaan yang tepat

- 1 **Masalah** — LIKE + facet di PostgreSQL tidak skala (≈ 3 dtk @ 1 juta).
- 2 **Akar** — satu model dipaksa melayani write **dan** search.
- 3 **CQRS** — pisahkan write (PostgreSQL) dari read (OpenSearch).
- 4 **Jembatan** — CDC (Debezium $\rightarrow$ Kafka), bersih, tanpa ubah penulis.
- 5 **Hasil** — hasil sama, kecepatan puluhan ms vs detik.
- 6 **Harga** — eventual consistency + kompleksitas. Pilih sesuai kebutuhan.

— TERIMA KASIH • Q&A

Database yang tepat untuk pekerjaan yang tepat.

CQRS membuat itu mungkin — tanpa mengorbankan sumber kebenaran.

► Repo: [ProgrammerZamanNow/cqrs-demo](#)

`podman compose up -d --build`

`make register` → buka :3000